

MATH 3302 Coding Theory

Lecturer: Barbara Maenhaut

Office location: 67-548

Office phone: 3365 3258

Email: bmm@maths.uq.edu.au

Consultation hours: Mondays 1-2
or by appointment

Reminder of the in-class coding
theory exercise on 6 May or 7 May
during your applied class.

Coding Theory Motivation

Coding $\neq$ Cryptography

Cryptography = keeping information secret

Coding = keeping information error-free

In cryptography, an encryption + decryption algorithm form a cipher (not a code).

We will not be concerned with secrecy at all in the coding part of the course. Instead we will be focused on accuracy.

Coding theory is concerned with the reliable and efficient transmission of information through unreliable channels. Coding theory began with a 1948 paper of Shannon titled A mathematical theory of communication.

Inclusion of redundancy in the transmitted information enables the detection and correction of introduced errors. In general, more redundancy means better error detection and correction, but less efficiency. The challenge of coding theory is to find optimal codes, in terms of balancing error detection and correction capacity against efficiency of transmission.

Some quotes (with introduced errors):

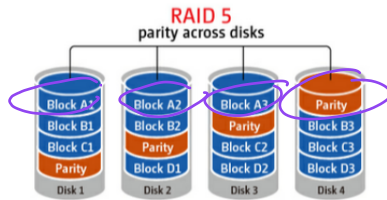
"So bt sure when you step, Step witj care ant great tact.
And remebber that liff's A Great Bakancing Act.
And will yoo succend? Yes! You wlll, indeed!"
– Dr. Seuss, Oh, The Places You'll Go!

"Think left and tuink right and tuinn low and thlnk hihh. Oh, tfe
ttinks you can lthink up iy only yot try!" – Dr. Seuss, "Oh, the
Thinks You Can Think"

"Ahe mone ehak tou mead, tht more bkings hou will kiow. Phe rone
ghat yop heatn, tea qore hfaces vou'vl go."
– Dr. Seuss, I Can Read With My Eyes Shut!

1.1.2 Example raid arrays

To increase reliability of storage, multiple devices can be linked together, using a *Redundant Array of Independent Disks* (raid). In a raid5 configuration, $1/4$ of the storage is set aside for redundancy. At the cost of losing $1/4$ of the storage space, even if one entire hard drive dies, no data¹ is lost. There are other variations, balancing robustness against efficiency.



In each position $A, B, C, D, E, \dots$, 3 bits are used for data and the 4th bit, called the *parity check bit*, is the sum of the others (mod 2). If 1 of the 4 bits cannot be read, its value can be deduced from the remaining 3: Given $w = x + y + z$, and 3 of x, y, z, w it is easy to find the value of the remaining bit.

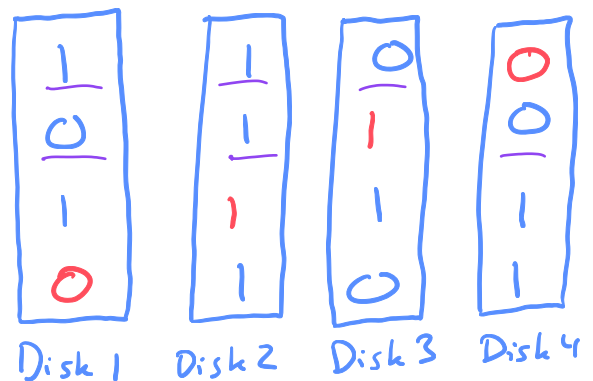
Example

$$(A_1, A_2, A_3) = (1, 1, 0)$$

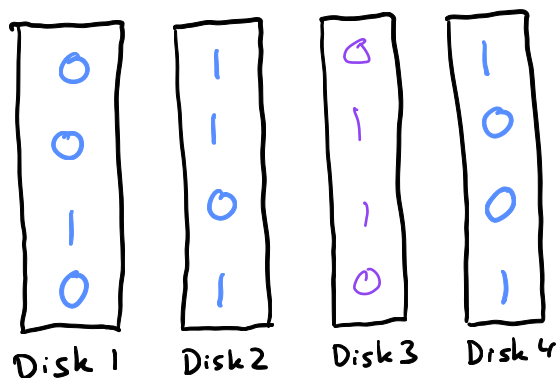
$$(B_1, B_2, B_3) = (0, 1, 0)$$

$$(C_1, C_2, C_3) = (1, 1, 1)$$

$$(D_1, D_2, D_3) = (1, 0, 1)$$



Exercise: In the RAID system below, disk 3 died and has been replaced. Determine the data that should be written to the replacement disk.



1.1.4 Example [DisplayPort] From extremetech & AnandTech:

The DisplayPort 1.4 standard was announced on 1/3/16, with support for video up to 7680×4320 resolution (8k) at 60 Hz.



DisplayPort 1.4 uses Display Stream Compression (dsc) to compress the signal. Corruption errors in the encoded data will wreck the decoded output in a way that users will immediately notice. An error in a compressed data chunk will cause decoding to fail or make the decoded result wrong over a large number of pixels, making the error far more noticeable.

For that reason, DisplayPort 1.4 adds error correction for the first time. Although error correction increases bandwidth requirements slightly, the additional, redundant data it carries allows for errors to be corrected, making dsc suitably reliable over DisplayPort connections. This is the key change that allows dsc to be used.

[Note that if data is compressed or encrypted, and then even a single bit error occurs, this may translate to a huge effect once the decompression/ decryption is applied.]

In summary: the primary goals of coding theory are to provide:

- (a) fast encoding of information;
- (b) easy transmission of encoded messages;
- (c) fast decoding of received message;
- (d) detection and correction of errors introduced in the channel;
- (e) maximum transfer of information per unit time.

These goals are not necessarily compatible! Goal (d) is where we will spend most of our discussion.

1.2. Informal Discussion

[This section is deliberately a bit vague. Precise definitions follow later.]

Data can be stored in many different formats. DNA can be viewed as data written with an alphabet with 4 letters C, A, G, T. English is written using a 26-letter alphabet.

However, throughout these notes, we will assume that whatever data we wish to send has been transformed into a binary sequence. Thus we assume our data is a (long) string of 0s and 1s.

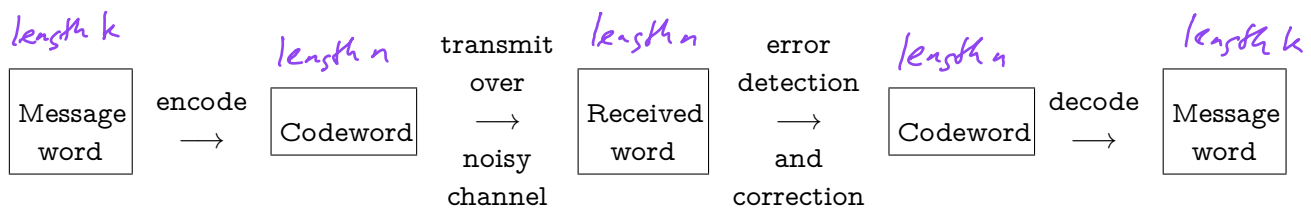
Since we do not know how long a message may be, we chop our long binary sequence into pieces of some fixed length k , and deal with each piece independently. So, we assume all the messages to be sent are binary sequences (=words = vectors) of length k . Such a binary word containing data to be transmitted is called a *message word*.

We wish to send our data across a *channel* : email, telephone line, cable, move from one hard drive to another etc. We assume the channel may be *noisy*. This means that *errors* may occur during transmission. An *error* occurs whenever a transmitted 0 is received as a 1, or a 1 is received as a 0.

To compensate for errors, we encode a message word of length k by (somehow) adding extra bits containing error correcting information. The result is called a codeword of length $n \geq k$. The set of all codewords is called a binary block code.

Further assume:

- ★ Any codeword of 0s and 1s of length n will be received as a word of length n containing 0s and 1s, not necessarily the same as the original codeword. No extra bits appear. No bits disappear. (If a bit cannot be read at all, just fill in that bit randomly; then this bit may well be an error.)
- ★ It is easy to identify the beginning of the first word, so we can identify each received word.



A simple way to detect/correct errors is to transmit the information multiple times. $k=2$

1.2.1 Example A 3-fold repetition code : Suppose we have four message words: 00, 01, 10, 11, and we encode them by listing them three times to get the four codewords 000000, 010101, 101010, 111111. The code is the set C of these four words $C = \{000000, 010101, 101010, 111111\}$. Here $k = 2$ and $n = 6$. Important: no other word except a codeword is ever transmitted.

We transmit these codewords of length 6 over a noisy channel. It is possible that error(s) occur during transmission, altering some 0s to 1s, or vice versa. After transmission, it is possible that we receive a word which is not in the code. In that case, we need to guess which of the codewords was most likely to have been sent.

Suppose the word 011101 is received. Since 011101 does not match any of the codewords and only codewords are ever transmitted, at least one error must have occurred. We have detected an error. We compare 011101 to each of the codewords to see which one was most likely to have been sent.

Codeword	0	0	0	0	0	0	# errors
Received	0	1	1	1	0	1	4
Codeword	0	1	0	1	0	1	
Received	0	1	1	1	0	1	1
Codeword	1	0	1	0	1	0	
Received	0	1	1	1	0	1	5
Codeword	1	1	1	1	1	1	
Received	0	1	1	1	0	1	2

Under the assumption that errors occur infrequently, it seems most likely(?) that the word actually sent was 010101. So we correct the received word $011101 \mapsto 010101$, corresponding to message word 01. We have corrected 1 error.

We cannot rule out the possibility that more errors occurred, and some other codeword was sent. In general, there is no guarantee in coding theory that we correctly recover the message that was

sent. We can only recover the codeword that was most likely sent given the received word. This will require some consideration of probabilities.

There are various points to clarify:

- (a) How much redundant information should be added, and in what form?
- (b) How many errors can we detect?
- (c) If errors do occur, how should we correct them, and recover a codeword?
- (d) How many errors can we correct?

Puzzle Bob is trying to work out Alice's birthday, so he can buy her a surprise present. First he wants to guess the day of the month of Alice's birthday $\in \{1, 2, \dots, 31\}$. He asks her a series of questions with yes/no answers.

- (a) Assuming Alice answers truthfully, how many questions (max) does Bob need? (Easy)
- (b) Suppose that Alice may lie on ≤ 1 of her answers, and tell the truth on the others. Now how many questions does Bob need? (Not so easy)
- (c) What has this to do with coding theory?

Binary sequence

1 00001
 2 00010
 3 00011
 4 00100
 ⋮
 31 11111

5 questions needed if Alice tells the truth.

$n = ?$

$k = 5$

1-error correcting

Background Material

Before we start coding theory formally, we review some background material.

2.1. Notation & Review of Arithmetic Modulo 2

Our messages will be represented by strings of 0s and 1s.

2.1.1 Definition Let $\mathbb{F}_2 = \{0, 1\}$. We define addition (denoted $+$) and multiplication on $\mathbb{F}_2$ by defining $1 + 1 = 0$. The full tables for addition and multiplication are:

$+$	0	1
0	0	1
1	1	0

$\bullet$	0	1
0	0	0
1	0	1

This is called the set of *integers modulo 2*, or *integers mod 2*, or the *finite field* with 2 elements. (Other notation: $\mathbb{Z}_2$, or $\mathbb{Z}/2\mathbb{Z}$.)

Note: if 0 and 1 represent false and true respectively, multiplication is logical AND, while addition is exclusive or, XOR.

The set

$$\mathbb{F}_2^n = \mathbb{F}_2 \times \cdots \times \mathbb{F}_2 = \{(x_1, x_2, \dots, x_n) \mid x_1, x_2, \dots, x_n \in \mathbb{F}_2\}$$

consists of all n -tuples $(x_1, \dots, x_n)$ with each $x_i \in \mathbb{F}_2$. These are vectors (strings/ words/ sequences), not binary numbers. We write $\mathbf{0}_n$ or just $\mathbf{0}$ for $(0, 0, \dots, 0) \in \mathbb{F}_2^n$. We write $\mathbf{1}_n$ or $\mathbf{1}$ for $(1, 1, \dots, 1) \in \mathbb{F}_2^n$. The individual entries x_i are sometimes called *bits*.

We can add (but not multiply) elements of $\mathbb{F}_2^n$:

$$(x_1, \dots, x_n) + (y_1, \dots, y_n) = (x_1 + y_1, \dots, x_n + y_n).$$

There is no “carry” when we add. We add in each coordinate (bit) independently.

We often omit the commas and brackets.
eg. We write 011100 instead of $(0, 1, 1, 1, 0, 0)$.

Exercise

- ① How many elements are there in $\mathbb{F}_2^n$? 2^n .
- ② True or false?
 - (a) $\forall v \in \mathbb{F}_2^n, v + v = \mathbf{0}$ *true*
 - (b) $\forall u, v, w \in \mathbb{F}_2^n, u + v = w \iff u = v + w$ *true*

2.1.2 Definition If $u = (u_1, u_2, \dots, u_n)$, and $v = (v_1, v_2, \dots, v_n) \in \mathbb{F}_2^n$ their dot product $u \cdot v$ is defined to be $\sum_{i=1}^n u_i v_i \in \mathbb{F}_2$. If $u \cdot v = 0 \in \mathbb{F}_2$ then u and v are said to be orthogonal or perpendicular. We write $u \perp v$.

Exercise:

- ① Find all vectors of $\mathbb{F}_2^3$ that are orthogonal to 100.
- ② Find all vectors of $\mathbb{F}_2^3$ that are orthogonal to 101.
- ③ Find all vectors of $\mathbb{F}_2^3$ that are orthogonal to 111.
- ④ What do you notice about your answers to ①, ②, ③?

$$\textcircled{1} \quad (a, b, c) \cdot (1, 0, 0) = a + 0 + 0 = a$$

$$\{000, 010, 001, 011\}$$

$\underline{0}$ is always in there
closed under addition

$$\textcircled{2} \quad (a, b, c) \cdot (1, 0, 1) = a + c$$

$$\{000, 010, 101, 111\}$$

2-dim subspaces

$$\textcircled{3} \quad (a, b, c) \cdot (1, 1, 1) = a + b + c$$

$$\{000, 110, 101, 011\}$$

2.1.5 **Definition** A finite sequence of 0s and 1s is called a word. The length of a word is the number of symbols (bits) in the word. Thus $\mathbb{F}_2^n$ is the set of words of length n .

We may define a word w as the concatenation of two shorter words u, v .

2.1.6 **Definition** $w = (u \mid v)$ means: w is the word consisting of all the bits of u followed by the bits of v .

2.1.7 **Example** If $u = 110$ and $v = 011$ then $(u \mid v) = 110011$.

2.1.8 **Definition** In $\mathbb{F}_2^n$ we let θ_j denote the word that has a 1 in position j , and 0s elsewhere.

2.1.9 **Example** In $\mathbb{F}_2^3$ we have words $\theta_1 = 100$, $\theta_2 = 010$, $\theta_3 = 001$.

2.1.10 **Definition** If $A, B \subseteq \mathbb{F}_2^n$ we write

$$A + B = \{a + b \mid a \in A, b \in B\}$$

Thus $A + B \subseteq \mathbb{F}_2^n$. If B consists of a single element $B = \{b\}$ we write $A + b$ instead of $A + \{b\}$.

We can also form polynomials with $\mathbb{F}_2$ -coefficients. The set of all such polynomials is denoted $\mathbb{F}_2[x]$. We add and multiply such polynomials in the normal way, using the addition and multiplication in $\mathbb{F}_2$ on the coefficients.

$$(\cancel{x^2} + 1)(\cancel{x} + \cancel{x^2}) = \cancel{x^3} + \cancel{x^4} + 1$$

Exercise: Expand and simplify $(x^4 + x^2 + 1)(x^3 + x)$, working in $\mathbb{F}_2[x]$.

$$= x^7 + \cancel{x^5} + \cancel{x^5} + \cancel{x^3} + \cancel{x^3} + x$$

$$= x^7 + x$$

$$(x+1)^2 = x^2 + 1$$

2.2. Distances and Weights

2.2.1 Lemma If v, w are words in $\mathbb{F}_2^n$ then $v + w$ is a word containing a 1 in exactly those places in which v and w differ, and a 0 in those places they agree. In particular, $v + w = \mathbf{0}$ iff $v = w$.

$$\begin{array}{ccccccc}
 v = & \dots & 1 & \dots & 1 & \dots & 0 & \dots & 0 \\
 w = & \dots & 1 & \dots & 0 & \dots & 1 & \dots & 0 \\
 \hline
 v+w = & & 0 & & 1 & & 1 & & 0
 \end{array}$$

2.2.2 Definition Let $v \in \mathbb{F}_2^n$ be a word of length n . The weight or Hamming weight of v , denoted $\|v\|$, is the number of occurrences of the bit 1 in v . Note that $0 \leq \|v\| \leq n$.

2.2.3 Lemma If $v \in \mathbb{F}_2^n$, then

$$(2.2.2) \quad \|v + \mathbf{1}_n\| = n - \|v\|$$

$\|v\|$ is the number of 1s in v . So the number of 1s in $v + \mathbf{1}_n$ is $n - \|v\|$.

2.2.5 Lemma

- (a) $\|u + v\| \leq \|u\| + \|v\|$
- (b) $\|u + v\| \equiv \|u\| + \|v\| \pmod{2}$
- (c) If $u \perp v$ then $\|u + v\| \equiv \|u\| + \|v\| \pmod{4}$.

Consider the four possible bit combinations for u and v .

$$\begin{array}{ccccccc}
 & & \text{A} & & \text{B} & & \text{C} & & \text{D} \\
 u = & \dots & 0 & \dots & 0 & \dots & 1 & \dots & 1 \\
 v = & \dots & 0 & \dots & 1 & \dots & 0 & \dots & 1 \\
 \hline
 u+v = & \dots & 0 & \dots & 1 & \dots & 1 & \dots & 0
 \end{array}$$

A # positions where u and v are both 0.

B
C
D

$$\|u\| = C + D$$

$$\|v\| = B + D$$

$$\|u+v\| = B + C$$

$$\|u\| + \|v\| = B + C + \underline{2D}$$

$$B + C \leq B + C + 2D$$

If $u \perp v$ then D is even $D = 2k$ for some $k \in \mathbb{Z}$

$$\begin{aligned}
 \|u\| + \|v\| &= B + C + 4k \\
 &= \|u+v\| + 4k
 \end{aligned}$$

2.2.6 Definition The *Hamming distance* $d(u, v)$ between two words $u, v \in \mathbb{F}_2^n$ is the number of places in which their bits differ.

By Lemma 2.2.1

$$(2.2.3) \quad d(u, v) = \|u + v\|.$$

$$\begin{array}{r} u = \dots 0 \dots 0 \dots 1 \dots 1 \\ v = \dots 0 \dots 1 \dots 0 \dots 1 \\ \hline u+v = \dots 0 \dots 1 \dots 1 \dots 0 \end{array}$$

2.2.8 Theorem If $x, y, z \in \mathbb{F}_2^n$ then

- (a) $d(x, y) \geq 0$
- (b) $d(x, y) = 0$ iff $x = y$
- (c) $d(x, y) = d(y, x)$ (symmetry)
- (d) $d(x, z) \leq d(x, y) + d(y, z)$ (the triangle inequality).

2.3. Floor and Ceiling

Revision from MATH1061 – read through the notes to make sure you remember this.

2.3.9 Lemma If $m \in \mathbb{Z}$ and $\theta \in \mathbb{R}$, then

$$(2.3.1) \quad \lfloor m - \theta \rfloor = m - \lceil \theta \rceil \quad \text{and} \quad \lceil m - \theta \rceil = m - \lfloor \theta \rfloor.$$

$\lfloor \emptyset \rfloor = 0 \quad \lceil \emptyset \rceil = 1$

Proof Write $\theta = n + \phi$ where $n \in \mathbb{Z}$ and $0 \leq \phi < 1$. If $\phi = 0$, then $\theta \in \mathbb{Z}$ and $m - \theta \in \mathbb{Z}$ so

$$\lfloor m - \theta \rfloor = \lceil m - \theta \rceil = m - \lceil \theta \rceil = m - \lfloor \theta \rfloor.$$

Otherwise, $0 < \phi < 1$ and

$$\lfloor m - \theta \rfloor = \lfloor m - n - \phi \rfloor = m - n - 1 = m - (n + 1) = m - \lceil \theta \rceil$$

and

$$\lceil m - \theta \rceil = \lceil m - n - \phi \rceil = m - n = m - \lfloor \theta \rfloor.$$